

Software Engineering I

Assignment 1

Fabian Gonzalez

September 8, 2026

1. The two GenAI programs that I used for this assignment are Copilot and Gemini.
2. I accessed both of them through a google search. Gemini was easier to access since it was already available on the browser without having to look it up. Copilot on the other hand had to be searched through the Windows pop-up.
3. Prompt used for both GenAI models:

Write a C program that picks a secret number between 1 and 10. The user gets up to 3 tries to guess the number.

Requirements:

- Use a fixed secret number (for example, 7).
- Use printf to prompt the user and scanf to read their guess.
- Use a while or for loop to let the user guess up to 3 times.
- Use if-else statements to check if the guess is too high, too low, or correct.
- Stop the loop immediately if the user guesses the correct number.
- Print a final win or lose message.

Example Output:

Guess a number between 1 and 10.

Attempt 1/3. Enter your guess: 5

Too low! Try again.

Attempt 2/3. Enter your guess: 8

Too high! Try again.

Attempt 3/3. Enter your guess: 7

Correct! You win!

4. Generated code from both GenAIs:

Copilot:

```
#include <stdio.h>
```

```
int main() {  
    int secretNumber = 7; // Fixed secret number  
    int guess;  
    int attempts;  
    int won = 0;  
  
    printf("Guess a number between 1 and 10.\n");
```

```

for (attempts = 1; attempts <= 3; attempts++) {
    printf("Attempt %d/3. Enter your guess: ", attempts);
    scanf("%d", &guess);

    if (guess == secretNumber) {
        printf("Correct! You win!\n");
        won = 1;
        break; // Stop the loop immediately
    }
    else if (guess < secretNumber) {
        printf("Too low! Try again.\n");
    }
    else {
        printf("Too high! Try again.\n");
    }
}

if (!won) {
    printf("Sorry, you lose! The secret number was %d.\n", secretNumber);
}

return 0;
}

```

Gemini:

```

#include <stdio.h>

int main() {
    int secret = 7;
    int guess;
    int max_attempts = 3;
    int won = 0;

    printf("Guess a number between 1 and 10.\n");

    for (int attempt = 1; attempt <= max_attempts; attempt++) {
        printf("Attempt %d/%d. Enter your guess: ", attempt, max_attempts);

        // Read user input
        if (scanf("%d", &guess) != 1) {
            printf("Invalid input. Exiting.\n");
            return 1;
        }

        // Check the guess
        if (guess == secret) {
            printf("Correct! You win!\n");
            won = 1;

```

```

        break; // Exit the loop immediately
    } else if (guess < secret) {
        if (attempt < max_attempts) {
            printf("Too low! Try again.\n");
        }
    } else {
        if (attempt < max_attempts) {
            printf("Too high! Try again.\n");
        }
    }
}

// Print lose message if maximum attempts reached without guessing correctly
if (!won) {
    printf("Out of attempts! You lose. The secret number was %d.\n", secret);
}

return 0;
}

```

5. Comparing the two in terms of correctness, execution time, space complexity, and maintainability

Correctness:

Copilot - Code executes as expected with a single or multidigit number. Code gives you an output if you do or do not manage to correctly guess the number. Code does not handle string inputs or symbols well. If used, the system will eat up the user's attempts and enforce a game over. Oddly enough, if the correct number is used in front of strings or symbols, it will not detect an error and assume you guessed correctly. Will repeat if the number is too high or too low even if they are out of attempts.

Gemini - Code executes as expected with a single or multidigit number. Correct outputs are given if you do or do not manage to correctly guess the number. Code handles string and symbol inputs as well. If a string or a symbol is detected, it immediately exits the program, preventing more attempts to be used. Oddly enough, if the correct number is used in front of strings or symbols, it will not detect an error and assume you guessed correctly.

Execution Time:

- Claude - Best-case scenario: $O(1)$. Worst-case scenario: $O(n)$
- Gemini - Best-case scenario: $O(1)$. Worst-case scenario: $O(n)$

Both programs fall under the same execution times. This is because in their best case scenarios, they only need one input. At their worst, they would need 3 inputs just for the user to guess correctly.

Space Complexity:

- Claude - Best-case scenario: $O(1)$. Worst-case scenario: $O(n)$
- Gemini - Best-case scenario: $O(1)$. Worst-case scenario: $O(n)$

Both programs contain the same space complexity. Despite being able to take in 3 inputs, the memory is not dependent on how many guesses you need, causing the space complexity to be $O(1)$ instead of $O(n)$.

Maintainability:

Claude - Code features less comments about how the code works and its printf and scanf are slow because more triggers are being called. Implementing a return value to the scanf would prevent more broken loops or unnecessary behavior from causing the program to behave slowly.

Gemini - Code features more comments about how the code functions but includes a `won = 0` variable which could be improved by replacing the 0 to a 1 or implementing a True or False. Neither of which would expand nor reduce space complexity and execution time, but it would help with making the code more readable to the users.

Both code programs do not take into account a minimum and maximum value for what number the user could guess. If the user guesses above or under, the programs should both shutdown as to reduce time and space complexity.

6. The program that I will use for the basis of my assignment will be the Gemini code. The reason being is that it is more thoroughly structured than Copilot's code and provides comments for what certain parts of the program do. The main thing being that the program is able to handle strings and symbols differently than Copilot which can still be improved upon.
7. I improved the correctness by establishing a minimum and maximum so that the user adheres to the rules and does not try to guess over or under the limits. I also tried to

I improved the execution time by also providing limits to what numbers can or cannot be guessed. Making sure that the user does I improved the space complexity by... I improved the maintainability by adding more comments and by testing the program repeatedly to look for more noticeable errors